

线段树

- 建树
- 单点加
- 单点赋值
- 区间加
- 区间赋值
- 区间和查询
- 区间最大值查询
- 区间最小值查询

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;
#define sz(a) int((a).size())
#define pii pair<int, int>
#define pll pair<ll, ll>
#define all(a) (a).begin(), (a).end()
const int inf = (1 << 30);
const ll INF = (1ll << 60);
const double PI = acos(-1);

struct SegTree{
    int n;
    vector<ll> a, sum, mx, mn, add, tag;
    vector<int> cov;

    SegTree(int _n = 0){
        if(_n) init(_n);
    }

    SegTree(const vector<ll>& _a){
        init(_a);
    }

    void init(int _n){
        n = _n;
        a.assign(n + 1, 0);
        sum.assign((n + 1) << 2, 0);
        mx.assign((n + 1) << 2, 0);
        mn.assign((n + 1) << 2, 0);
        add.assign((n + 1) << 2, 0);
        tag.assign((n + 1) << 2, 0);
        cov.assign((n + 1) << 2, 0);
    }
};
```

```

}

void init(const vector<ll>& _a){
    a = _a;
    n = sz(a) - 1;
    sum.assign((n + 1) << 2, 0);
    mx.assign((n + 1) << 2, 0);
    mn.assign((n + 1) << 2, 0);
    add.assign((n + 1) << 2, 0);
    tag.assign((n + 1) << 2, 0);
    cov.assign((n + 1) << 2, 0);
    build(1, 1, n);
}

void pull(int p){
    sum[p] = sum[p << 1] + sum[p << 1 | 1];
    mx[p] = max(mx[p << 1], mx[p << 1 | 1]);
    mn[p] = min(mn[p << 1], mn[p << 1 | 1]);
}

void apply_add(int p, int l, int r, ll v){
    sum[p] += v * (r - l + 1);
    mx[p] += v;
    mn[p] += v;
    if(cov[p]) tag[p] += v;
    else add[p] += v;
}

void apply_set(int p, int l, int r, ll v){
    sum[p] = v * (r - l + 1);
    mx[p] = mn[p] = v;
    tag[p] = v;
    cov[p] = true;
    add[p] = 0;
}

void push(int p, int l, int r){
    if(l == r) return;

    int mid = (l + r) >> 1;
    if(cov[p]){
        apply_set(p << 1, l, mid, tag[p]);
        apply_set(p << 1 | 1, mid + 1, r, tag[p]);
        cov[p] = false;
    }
    if(add[p]){
        apply_add(p << 1, l, mid, add[p]);
        apply_add(p << 1 | 1, mid + 1, r, add[p]);
        add[p] = 0;
    }
}

```

```

}

void build(int p, int l, int r){
    if(l == r){
        sum[p] = mx[p] = mn[p] = a[l];
        return;
    }

    int mid = (l + r) >> 1;
    build(p << 1, l, mid);
    build(p << 1 | 1, mid + 1, r);
    pull(p);
}

void range_add(int p, int l, int r, int x, int y, ll v){
    if(x <= l && r <= y){
        apply_add(p, l, r, v);
        return;
    }

    push(p, l, r);
    int mid = (l + r) >> 1;
    if(x <= mid) range_add(p << 1, l, mid, x, y, v);
    if(y > mid) range_add(p << 1 | 1, mid + 1, r, x, y, v);
    pull(p);
}

void range_set(int p, int l, int r, int x, int y, ll v){
    if(x <= l && r <= y){
        apply_set(p, l, r, v);
        return;
    }

    push(p, l, r);
    int mid = (l + r) >> 1;
    if(x <= mid) range_set(p << 1, l, mid, x, y, v);
    if(y > mid) range_set(p << 1 | 1, mid + 1, r, x, y, v);
    pull(p);
}

ll query(int p, int l, int r, int x, int y){
    if(x <= l && r <= y){
        return sum[p];
    }

    push(p, l, r);
    ll ans = 0;
    int mid = (l + r) >> 1;
    if(x <= mid) ans += query(p << 1, l, mid, x, y);
    if(y > mid) ans += query(p << 1 | 1, mid + 1, r, x, y);
}

```

```

        return ans;
    }

    ll qmax(int p, int l, int r, int x, int y){
        if(x <= l && r <= y){
            return mx[p];
        }

        push(p, l, r);
        ll ans = -INF;
        int mid = (l + r) >> 1;
        if(x <= mid) ans = max(ans, qmax(p << 1, l, mid, x, y));
        if(y > mid) ans = max(ans, qmax(p << 1 | 1, mid + 1, r, x, y));
        return ans;
    }

    ll qmin(int p, int l, int r, int x, int y){
        if(x <= l && r <= y){
            return mn[p];
        }

        push(p, l, r);
        ll ans = INF;
        int mid = (l + r) >> 1;
        if(x <= mid) ans = min(ans, qmin(p << 1, l, mid, x, y));
        if(y > mid) ans = min(ans, qmin(p << 1 | 1, mid + 1, r, x, y));
        return ans;
    }

    void add_point(int x, ll v){
        range_add(1, 1, n, x, x, v);
    }

    void set_point(int x, ll v){
        range_set(1, 1, n, x, x, v);
    }

    void add(int l, int r, ll v){
        range_add(1, 1, n, l, r, v);
    }

    void setv(int l, int r, ll v){
        range_set(1, 1, n, l, r, v);
    }

    ll query(int l, int r){
        return query(1, 1, n, l, r);
    }

    ll qmax(int l, int r){

```

```

        return qmax(1, 1, n, l, r);
    }

    ll qmin(int l, int r){
        return qmin(1, 1, n, l, r);
    }
};

void work(){
    int n, q;
    cin >> n >> q;

    vector<ll> a(n + 1);
    for(int i = 1; i <= n; ++i){
        cin >> a[i];
    }

    SegTree seg(a);

    while(q--){
        int op;
        cin >> op;

        if(op == 1){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            seg.add(l, r, v);           // 区间加
        }else if(op == 2){
            int l, r;
            ll v;
            cin >> l >> r >> v;
            seg.setv(l, r, v);         // 区间赋值
        }else if(op == 3){
            int x;
            ll v;
            cin >> x >> v;
            seg.add_point(x, v);       // 单点加
        }else if(op == 4){
            int x;
            ll v;
            cin >> x >> v;
            seg.set_point(x, v);       // 单点赋值
        }else if(op == 5){
            int l, r;
            cin >> l >> r;
            cout << seg.query(l, r) << '\n'; // 区间和
        }else if(op == 6){
            int l, r;
            cin >> l >> r;

```

```
        cout << seg.qmax(l, r) << '\n'; // 区间最大值
    }else{
        int l, r;
        cin >> l >> r;
        cout << seg.qmin(l, r) << '\n'; // 区间最小值
    }
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int T = 1;
    cin >> T;
    while(T--) work();
    return 0;
}
```